

ISCTE

CDB_05161

Cryptography for Blockchain Developers
Practical Exercises and Secure Mini
Blockchain

Student	Sudip Phuyal
Student Number	126340
Submitted to	Prof. Joao Carlos Ferreira
Institution	Iscte, University Institute of Lisbon
Repository	github.com/sudipphuyal/crypto-blockchain-assignment
Submission Date	7 July 2026

Course assignment report

Abstract

This report documents the implementation and evaluation of the practical work for the course Cryptography for Blockchain Developers. The work covers seven applied exercises: hash functions and Proof-of-Work; elliptic-curve keys and address derivation; ECDSA signing and nonce reuse; Merkle trees and SPV proofs; zero-knowledge proof concepts and the Schnorr identification protocol; hierarchical deterministic wallets; and selected cryptographic attacks and defences. The final project integrates the principal primitives into a secure mini blockchain written in Python. It includes secp256k1 wallets, signed transactions, a Merkle root, simplified Proof-of-Work, an account-based ledger, chain validation and SPV-style inclusion proofs. Execution evidence stored in the public repository shows that the end-to-end workflow completed as expected and that 17 automated tests passed.

Keywords: Blockchain, cryptography, SHA-256, ECDSA, secp256k1, Merkle tree, Proof-of-Work, BIP32, BIP39, SPV.

Repository and evidence

The complete source code, exercise scripts, execution logs, test suite and commit history are publicly available at github.com/sudipphuyal/crypto-blockchain-assignment. The repository includes dedicated folders for exercises, outputs and tests, alongside the final `blockchain.py` and `main.py` files.

Report structure

- 1. Introduction and approach
- 2. Practical exercises
- 3. Final project: Secure Mini Blockchain
- 4. Verification and test evidence
- 5. Discussion and limitations
- 6. AI assistance declaration
- References and appendix

1. Introduction and approach

Blockchain systems rely on a small set of interdependent cryptographic primitives. **Hash functions** provide tamper-evident commitments; **public-key cryptography** supports identity and signatures; **Merkle trees** enable scalable inclusion verification; and **consensus mechanisms** decide which valid transactions become part of a shared history. The practical work was organised progressively: individual scripts were first built and executed for each primitive, after which the reusable components were integrated into a final blockchain prototype.

The coursework handout requires practical code and reflection rather than a purely theoretical discussion. Accordingly, each exercise records observable outputs and explains the security property being demonstrated. The final project follows the prescribed sequence of creating wallets, signing transactions, mining a block, checking the chain, constructing an SPV proof and detecting a post-mining alteration [1].

Stage	Implementation approach	Evidence retained
Exercises 1-7	Independent Python scripts, each focused on a defined primitive or attack/defence concept.	Saved console output under outputs/.
Final integration	Reusable Wallet, Transaction, MerkleTree, Block and Blockchain classes.	blockchain.py and main.py.
Verification	Automated tests plus a complete end-to-end execution.	tests/test_blockchain.py, test_results.txt and final_demo.txt.

Stage	Implementation approach	Evidence retained
Version control	Public Git repository with incremental commits.	GitHub repository and commit history.

2. Practical exercises

The following sections summarise the implementation choices, results and learning outcomes for the seven practical exercises. Full code and unabridged console outputs are available in the public repository.

2.1 Exercise 1 - Hash functions and Proof-of-Work

Implementation. The script calculates SHA-256 for “Vinho do Porto”, “vinho do porto” and the empty string; counts the bit differences produced by a one-character modification; mines a simplified block header at three difficulty levels; applies double SHA-256 to a simulated transaction; derives an Ethereum-style address from Keccak-256; and reproduces the Bitcoin genesis-block hash.

Observed result	Interpretation
138 of 256 bits changed (53.91%)	A one-character change produced near-half output variation, illustrating the avalanche effect.
PoW difficulty 1, 2 and 3 completed	The nonce search effort increased with the number of leading hexadecimal zeroes, while verification remained a single hash operation.
Genesis hash matched the known value	The double-SHA-256 calculation reproduced 000000000019d668...a8ce26f.

Learning outcome. Hashing provides a fixed-size, deterministic commitment to data. Once a block is changed, its hash and the previous-hash link in each later block no longer match; an attacker would have to recompute the altered block and all subsequent blocks.

2.2 Exercise 2 - Elliptic-curve keys and addresses

Implementation. A fresh test-only secp256k1 key pair was generated. The implementation produced a 32-byte private key, a 64-byte uncompressed public-key body, a 33-byte compressed public key, a compressed mainnet WIF representation, a Bitcoin P2PKH address and an Ethereum address. A short vanity-address search for the prefix “1A” was also executed.

Result. All structural checks passed. The vanity search found a test address with the “1A” prefix in 13 attempts; the expected mean is probabilistic and the simple approximation used in the exercise is about 58 attempts. The exercise distinguishes the practical forward operation $K = kG$ from the computationally infeasible reverse elliptic-curve discrete logarithm problem.

2.3 Exercise 3 - ECDSA signatures and nonce reuse

Implementation. Alice signed a simulated “Alice sends 1 BTC to Bob” message using ECDSA over secp256k1, and the DER-encoded signature was verified with Alice’s public key. The amount was then modified from 1 BTC to 2 BTC, after which verification failed. A separate controlled demonstration reused a single nonce k for two test signatures.

Result. The original message validated and the modified message did not. In the controlled nonce-reuse scenario, both signatures shared the same r value. The script recovered the reused nonce and the temporary private scalar, then verified that the recovered scalar generated the same public key. This demonstrates why ECDSA nonce reuse completely compromises the corresponding signing key.

```
k = (z1 - z2) * (s1 - s2)^(-1) mod n
privateKey = (s1 * k - z1) * r^(-1) mod n
```

2.4 Exercise 4 - Merkle trees and SPV inclusion proofs

Implementation. A binary Merkle tree was created for four simulated transactions. The script calculated the leaf hashes, parent hashes and final root; altered one transaction to demonstrate root invalidation; generated a proof for “Eve -> Frank”; and verified the proof by reconstructing the root using sibling hashes and their left/right position.

Result. Changing “Carol -> Dave” to “Carol -> Mallory” produced a different Merkle root. The SPV proof for the third transaction was valid; the same proof failed for a modified transaction. The scale calculation showed that a proof for one million transactions needs approximately $\text{ceil}(\log_2(1,000,000)) = 20$ sibling hashes, or about 640 bytes of SHA-256 hash data plus direction metadata.

2.5 Exercise 5 - Zero-knowledge proofs and Schnorr identification

Implementation. The exercise simulated the Alibaba cave protocol for 10 and 30 rounds, implemented a toy Schnorr identification protocol over deliberately small parameters, tested a random guessed response, and provided a conceptual treatment of shielded transactions, zk-SNARKs, zk-STARKs, zkRollups and Bulletproofs.

Result. The theoretical success probability for a dishonest cave prover was $1/2^{10} = 0.09765625\%$ after 10 rounds and $1/2^{30}$ approximately 0.000000093132% after 30 rounds. In the Schnorr run, the verification equation held for the honest prover and failed for a random fake response. The code deliberately uses tiny values only so that the protocol steps can be inspected; it is not production cryptography.

```
Public value: Y = g^x mod p
Commitment:  t = g^r mod p
Response:    s = r + c*x mod q
Verification: g^s mod p = t * Y^c mod p
```

2.6 Exercise 6 - HD wallets, BIP32, BIP39 and BIP44

Implementation. A 12-word temporary BIP39 mnemonic was generated and immediately hidden from saved output. It was converted into a 512-bit seed using PBKDF2-HMAC-SHA512 with 2048 iterations. The project then derived BIP32 master private-key material and chain code, five Bitcoin addresses at m/44'/0'/0'/0/index, five Ethereum addresses at m/44'/60'/0'/0/index, and a Bitcoin account xpub at m/44'/0'/0'.

Result. The seed was 64 bytes, both master components were 32 bytes, all address derivation checks passed, determinism was confirmed, and the xpub was generated while remaining hidden from the public log. A 12-word mnemonic provides 128 bits of entropy plus a four-bit checksum; the script estimates a full 2^{128} search at 10^{12} guesses per second as approximately $1.078e+19$ years.

2.7 Exercise 7 - Cryptographic attacks and defences

Implementation. The final exercise used a deliberately weak 16-bit mini hash to demonstrate the birthday effect; implemented a parameterised majority-hashrate cost model without renting capacity or interacting with any network; modelled replay risk with and without a chain identifier; and implemented a hash-based commit-reveal check for a hidden token purchase action.

Demonstration	Recorded result
Birthday collision	Two distinct 16-byte inputs produced the same 16-bit mini-hash after 282 attempts; the rough birthday scale is $\text{sqrt}(2^{16}) = 256$.
Replay protection	A legacy conceptual payload verified in both chain scenarios; a payload bound to chainId=1 failed for chainId=61.
Commit-reveal	The correct revealed action and salt verified; an altered action failed verification.
Majority-hashrate model	Implemented as an economic formula only. No rented hash rate, mining service or live

Demonstration	Recorded result
	network interaction was used.

Learning outcome. The exercise links abstract security properties to realistic blockchain threats: truncated output makes collisions attainable; chain identifiers reduce replay risk; commit-reveal can mitigate some front-running situations; and a majority of Proof-of-Work capacity can reorder recent history but cannot forge other users' signatures.

3. Final project - Secure Mini Blockchain

The final project integrates the earlier primitives into a self-contained Python implementation. It uses an educational account-based model rather than Bitcoin's UTXO model. All monetary values are integer units, and the network/consensus layer is intentionally replaced with a single-process simulation.

3.1 Architecture

Wallet	Signed Transaction	Pending Pool	Block + Merkle Root	Proof-of-Work Mining	Chain + SPV Proof
Create keys -> Sign and validate -> Queue -> Commit transactions -> Find nonce -> Verify inclusion					
Component	Responsibilities implemented				
Wallet	Creates a secp256k1 test key pair, derives a Bitcoin P2PKH-style address, signs deterministic DER-encoded ECDSA messages and verifies signatures.				
Transaction	Stores sender, recipient, integer amount, timestamp, nonce, public key, signature and transaction ID; verifies that the sender address binds to the public key and that the signature protects the canonical payload.				
MerkleTree	Builds a binary tree over transaction IDs, duplicates the final leaf when a level is odd, calculates the root and creates/verifies SPV proof steps.				
Block	Contains index, previous hash, timestamp, transactions, Merkle root, nonce and hash; mines a hash with the required leading-zero prefix and validates the header, transactions, root and PoW.				
Blockchain	Creates the genesis allocation, maintains the pending pool, checks available balances, issues a fixed mining reward, validates links/rewards/balances/duplicate IDs and retrieves confirmed SPV proofs.				

3.2 Security controls

Control	How it is used in the prototype	Security purpose
Canonical serialisation	Transaction and block data are serialised with deterministic JSON before hashing/signing.	Prevents ambiguity in the bytes protected by a signature or hash.
ECDSA signature	A transfer is signed by the sender wallet and checked against the supplied public key and derived address.	Detects tampering and requires control of the sender key.
Merkle root	The block header commits to the Merkle root derived from transaction IDs.	Makes transaction-set changes detectable and supports inclusion proofs.
Proof-of-Work	Mining searches for a SHA-256 header hash with three leading hexadecimal zeroes.	Makes block creation computationally more expensive than validation in this simulation.
Balance replay	The full confirmed chain is replayed during validation; each transfer must have sufficient funds at its position.	Rejects overspending and preserves sequential state consistency.
Chain linking	Each non-genesis block contains the preceding block hash.	Makes modification of earlier history propagate to later blocks.

3.3 End-to-end execution

The final demonstration follows the required workflow below. Its full console output is retained as `outputs/final_demo.txt` in the repository.

Step	Action	Observed result
1	Create Alice, Bob and Miner wallets.	Three test-only P2PKH-style addresses were generated.
2	Create genesis block and allocate Alice 100 units.	Genesis hash satisfied the difficulty; chain validation returned True.
3	Alice signs a transfer of 10 units to Bob.	Signature validation returned True; transaction entered the pending pool.
4	Bob signs a transfer of 3 units to Alice.	The pending ordering allows Bob to spend the incoming 10 units in the same candidate block.
5	Miner mines the reward and both pending transfers.	Block index 1 was mined with a hash beginning 000; the Merkle root and block validation passed.
6	Replay balances and validate the complete chain.	Alice = 93, Bob = 7 and Miner = 5; full-chain validation returned True.
7	Generate SPV proof for Alice-to-Bob transfer.	The proof reconstructed the recorded Merkle root and returned True.
8	Change Alice-to-Bob amount from 10 to 999 after mining.	Complete chain validation returned False, detecting the tampering.

```
Recorded final execution evidence
Alice balance: 93
Bob balance:   7
Miner balance: 5
Complete chain valid: True
SPV proof valid: True
Chain valid after tampering: False
```

4. Verification and test evidence

The implementation was checked through focused unit tests and a separate full demonstration. The recorded pytest result is “17 passed in 0.16s”. The tests cover wallet/address formation, valid and altered signatures, Merkle proof validity, incorrect proof rejection, odd-leaf handling, block mining, nonce and previous-hash tampering, genesis validation, pending-order balance handling, insufficient-balance rejection, chain tampering and a confirmed-transaction SPV proof.

Test area	Examples of checks
Wallet and transaction	Address format; signed transfer validity; signature failure after changing amount; sender address/public-key binding.
Merkle and SPV	Correct inclusion proof; invalid proof for a different transaction; changed Merkle root after altering a transaction; odd-numbered leaves.
Block and PoW	Mined hash satisfies difficulty; invalid after transaction, nonce or previous-hash modification.
Blockchain state	Genesis balance; balance-aware pending ordering; insufficient-balance rejection; reward handling; chain validity; SPV proof of a confirmed transfer.

Recorded test result. The repository’s saved test result reports “17 passed in 0.16s”. The full final demonstration

separately confirms the expected balances, valid SPV proof and tamper detection.

5. Discussion and limitations

The project demonstrates the relationships among the primitives rather than attempting to replicate a production blockchain. The main achievement is the traceable security chain: a wallet signs a deterministic transaction payload; the transaction ID becomes a Merkle leaf; the root is committed into the block header; Proof-of-Work seals the header; and each block links to its predecessor. The end-to-end tamper demonstration shows that changing transaction content breaks both signature validity and the committed Merkle root, so full-chain validation fails.

Limitation	Implication / future extension
Single-process simulation	There are no peers, network propagation, fork choice or distributed consensus. A future version could introduce multiple nodes and a defined consensus protocol.
Educational PoW difficulty	Three leading hexadecimal zeroes are suitable for demonstration only, not for adversarial resistance.
Account-based model	Balances are replayed from the chain. Bitcoin uses the UTXO model, which would require input/output references and double-spend checks at the output level.
No persistent storage	The chain lives in memory during execution. A persistent format, database or state snapshot could be added.
Simplified transaction format	The prototype uses deterministic JSON and test addresses. Real networks use protocol-specific serialisation, fee rules, scripting/VM semantics and network validation.
Toy ZKP exercises	The Schnorr and shielded-transaction demonstrations are conceptual; they are not zero-knowledge proof systems suitable for deployment.

Despite these constraints, the project is useful as a coherent instructional artifact because every claimed property is supported by a dedicated script, stored output or automated test. The implementation deliberately avoids real wallet credentials: mnemonics, seeds, private keys, WIF values and xpubs were hidden from the saved public outputs.

6. Conclusion

The practical work achieved the intended progression from foundational cryptographic concepts to an integrated blockchain prototype. The seven exercises demonstrated hashing, elliptic-curve key generation, digital signatures, Merkle proofs, zero-knowledge concepts, hierarchical wallet derivation and common blockchain attack patterns. The final Python mini blockchain then combined those elements into a testable system with signed transactions, Proof-of-Work, Merkle commitments, account validation and SPV verification. The final execution produced the expected balances and correctly rejected a tampered chain. The public repository provides the code, outputs and commit history required to review or reproduce the work.

7. AI assistance declaration

Declaration. AI tools were used for conceptual explanations, code review, and debugging. All code was executed, tested, reviewed, and understood by the student.

The use of AI was limited to learning support and iterative review. The submitted implementation was run locally, the outputs were examined, the test suite was executed, and the final repository was organised and committed by the student.

Appendix A - Reproduction commands

The following commands reproduce the final demonstration and the test evidence from the repository root after the virtual environment has been activated and dependencies have been installed.

```
python -m pip install -r requirements.txt
python -m pytest -q
python main.py

# Re-run individual exercises
python exercises/01_hashes.py
python exercises/02_ecc_wallets.py
python exercises/03_ecdsa_signatures.py
python exercises/04_merkle_tree.py
python exercises/05_zkp_schnorr.py
python exercises/06_hd_wallets.py
python exercises/07_crypto_attacks.py
```

Appendix B - Submission inventory

Item	Location / purpose
Source code	Public GitHub repository: exercises/, blockchain.py, main.py and tests/.
Exercise evidence	outputs/01_hashes.txt through outputs/07_crypto_attacks.txt.
Final-project evidence	outputs/final_demo.txt and outputs/test_results.txt.
Dependencies	requirements.txt.
Report source	report.md and this editable DOCX report.